

DATA FUSION AND TRACKING OF COMPLEX TARGET MANEUVERS WITH A SIMPLEX-TRAINED NEURAL NETWORK-BASED ARCHITECTURE

Yee Chin Wong and Malur K. Sundareshan
Department of Electrical and Computer Engineering
University of Arizona
Tucson, AZ 85721-0104
Tel: (520) 621-2953; Fax: (520) 621-8076

ABSTRACT

The capabilities of trained neural networks to perform fusion of data collected from dissimilar sensors employed in target surveillance and tracking environments facilitate an attractive framework for developing advanced target tracking architectures. In this paper we shall present a novel scheme that employs an integration of a multilayer neural network trained with features extracted from multi-sensor data and a Kalman filter that yields a reliable tracking algorithm capable of following even non-cooperative targets executing complex evasive maneuvers. A learning strategy based on a simplex optimization algorithm that seeks the global minimum of the training error and a progressive network growing procedure are employed to develop the required capabilities underlying the desirable tracking performance delivered by the neural network tracking algorithm. Some representative performance validation studies are given in the form of tracking experiments involving targets executing not only straight line acceleration maneuvers but also complex turns in environments characterized by severe noise and clutter. A fundamental characteristic that deserves emphasis in the proposed target tracking architecture is the role of the neural network in performing data fusion and in providing assistance to a simple linear Kalman filter for tracking the maneuvering target, which provides an intelligent way of implementing an overall nonlinear tracking filter without any attendant increases in computational complexity.

1. INTRODUCTION

The ability of neural networks to fuse information from different sensors to assist in simple implementations of target surveillance and tracking algorithms is being appreciated in the very recent times [1,2]. For tracking non-cooperative targets executing complex evasive maneuvers, this capability affords the design of simple target tracking architectures by integrating a trained multilayer neural network with a simple tracking filter that provides estimates of target position and velocity by processing measurements from sensors deployed in a ground-based surveillance station, or on an airborne or

spaceborne reconnaissance platform, or in a tactical missile seeker.

Surveillance and tracking algorithms presently deployed in military or in air traffic control environments typically use information from only one sensor, such as a Track-While-Scan (TWS) radar. Although additional data such as Doppler and/or various forms of imagery data (infra-red, millimeterwave, electro-optic, etc.) are typically collected, utilizing these in the same signal processing channel to obtain improved detection, classification and track maintenance performance is highly challenging. A major limitation that precludes a simple integration of these additional data forms is the resulting complexity of the needed processing. It is quite well known that while simple linear processing algorithms employing a Kalman filter for target state estimation can be synthesized for processing radar data, inclusion of a different form of data (image or image-format data, for instance) will require a nonlinear processing method (such as an extended Kalman filter algorithm) [3]. The enormous processing complexity in the latter case could render the implementation impractical due to the real-time processing requirements underlying the tracking function and the need to keep up with the rapid target motions during the maneuvers. Consequently, an intelligent architecture that facilitates successful fusion of the diverse data forms to result in improved tracking performance in the face of complex target maneuvers is highly desirable.

In this paper we shall present a novel neural network-based architecture that provides a mechanism for fusing target position measurements from a TWS radar with data from other types of sensors in order to track target maneuvers executed in environments characterized by severe noise and clutter. This architecture, shown in Fig. 1, subjects the data stream coming from each sensor to a feature extraction operation, which is in turn followed by an integration of the features by a trained neural network. The neural network processing is aimed at obtaining a fused decision for classifying the maneuver type and for estimating the salient maneuver parameters which are in turn handed over to the tracking filter to perform the required state estimation. The performance of the overall

tracking scheme depends critically on the training provided to the neural network in order to permit an association to be made rapidly between the features extracted from the sensor measurements and target maneuver parameters. Since the received training is conducted off-line, significant benefits in developing real-time tracking capabilities are possible. Optimization of the overall training environment, however, requires (a) an appropriate selection of the feature vectors, (b) a careful design of the neural network architecture and learning algorithm, and (c) an efficient target dynamical model for generating the required training data and for implementation of the tracking filter. In this paper, we shall briefly describe some aspects of this study that are of particular interest to the neural network community. A validation of the proposed architecture is given in the form of tracking experiments involving targets executing complex maneuvers that involve high acceleration pull-offs and rapid turns.

2. DESCRIPTION OF TRACKING ARCHITECTURE

The basic building blocks of the tracking scheme shown in Fig. 2 are the neural network and the Kalman filter. The neural network accepts as inputs a set of features extracted from the sensor data and is trained to output estimates of a set of maneuver parameters characterizing the target maneuver that is represented in the feature set. Since features abstracted from the measurements obtained from dissimilar sensors are typically used as inputs to the neural network, the processing of data by the network implements a feature integration process and thus performs sensor fusion. The neural network outputs are fed to a Kalman filter which implements a recursive state estimation algorithm based on a linear model of the target dynamics. For the sake of illustration of specific details regarding the features extracted and the training conducted with these features, Fig. 2 depicts a fusion environment comprising of a range radar and a Doppler radar. It is to be emphasized that this is only for simplicity in discussing the details and will not limit the type of sensor that may be brought in to provide target measurements. Some essential details of this architecture that highlight the role of the neural network in recognizing different types of maneuvers executed by the target will be briefly outlined in this section.

2.1 Equivalent Velocity Tracking Model for Target Motion

One important aspect of the architecture shown in Fig. 2 that deserves special emphasis is that during the learning phase the neural network will be trained in conjunction with the Kalman filter which performs simple linear processing of the input data to yield target state estimates. The design of the Kalman filter, however, requires a reasonably accurate motion model for the target during the maneuvers. A standard target motion model that has been popularly used in the design of tracking

algorithms [4] is the "acceleration input model", which for a 2-dimensional tracking scenario uses four state variables (position and velocity in x- and y-directions) and two input variables (acceleration components in x- and y-directions). While this model serves to represent target motions quite accurately (assuming that a sufficiently fast sampling of data is employed) during maneuvers that involve accelerations along a longitudinal flight path (i.e. acceleration in the direction of the velocity vector), considerable errors are possible when the target maneuver involves rapid turns. Some previous studies that have addressed the problem of tracking target turns [4] have therefore recommended employing a different motion model, such as a "coordinated turn model" that explicitly uses a turn rate parameter in the description.

The use of different motion models to capture target motions during different maneuvers, while simple in concept, has a major impact on the neural network training complexity. In effect, this has the potential of increasing the number of neural networks to be trained if the class of expected maneuvers includes different types such as longitudinal accelerations, coordinated turns, non-coordinated turns, etc. The training complexity can be considerably reduced by the use of a new motion model which uses "equivalent velocity variables" $v_{xeq}(k)$ and $v_{yeq}(k)$ that relate two successive positions along the target motion path $\{x(k), y(k)\}$ and $\{x(k+1), y(k+1)\}$ by

$$x(k+1) = x(k) + T v_{xeq}(k); \quad y(k+1) = y(k) + T v_{yeq}(k),$$

T denoting the sampling period used. The resulting motion model can be briefly stated as

$$X(k+1) = FX(k) + G\delta(k) \quad (1)$$

where $X^T(k) = [x(k) \quad \dot{x}(k) \quad y(k) \quad \dot{y}(k)]$ is the state vector comprising of the position and velocity variables,

$\delta^T(k) = [\delta_x(k) \quad \delta_y(k)]$ with $\delta_x(k) = v_{xeq}(k) - \dot{x}(k)$ and $\delta_y(k) = v_{yeq}(k) - \dot{y}(k)$, and the matrices F and G being given by

$$F = \begin{bmatrix} 1 & T & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & T \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad G = \begin{bmatrix} T & 0 \\ 2 & 0 \\ 0 & T \\ 0 & 2 \end{bmatrix}.$$

The motion model given by (1) is referred to as "equivalent velocity tracking model" (EVTM) in the further discussion. This model provides a mechanism for implementing a Kalman filter in conjunction with which a single neural network needs to be trained for recognizing different maneuver types.

2.2 Design of Kalman filter

The Kalman filter implementation is based on the EVTm, which is rewritten in order to account for the presence of a process noise component as

$$X(k+1) = FX(k) + G\delta(k) + \gamma(k) \quad (2)$$

where $\gamma(k)$ represents the process noise modeled as a white Gaussian noise sequence with zero mean and covariance Q . This, together with the observation sequence

$$z(k) = HX(k) + \omega(k) \quad (3)$$

where $z(k)$ is the measurement of the target position coordinates (obtained from the TWS radar, for instance), $\omega(k)$ is the measurement noise modeled as a white Gaussian noise sequence with zero mean and covariance R , and

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix},$$

leads to a recursive algorithm to generate the minimum MSE estimates of the target state vector $\hat{X}(k)$. This algorithm is briefly outlined in the following steps (one may refer to [6] for details).

(i) One-step Prediction -

$$\hat{X}(k/k-1) = F\hat{X}(k-1/k-1) + G\hat{\delta}(k-1) \quad (4)$$

(ii) Filtering -

$$\hat{X}(k/k) = \hat{X}(k/k-1) + K(k)[z(k) - H\hat{X}(k/k-1)] \quad (5)$$

(iii) Gain Computation -

$$K(k) = P(k/k-1)H^T[HP(k/k-1)H^T + R]^{-1} \quad (6)$$

(iv) Covariance Updating -

$$P(k/k-1) = FP(k-1/k-1)F^T + GQG^T \quad (7)$$

$$P(k/k) = [I - K(k)H]P(k/k-1) \quad (8)$$

The estimate of the velocity deviation vector $\hat{\delta}$ needed for implementation of (4) is developed by a three-layer neural network which is trained with a set of seven feature vectors derived from the measurements from a TWS radar and a Doppler radar. The seven features $\tilde{v}_1 - \tilde{v}_7$ used for training are listed briefly as under:

$$\tilde{v}_1(k) = \frac{\tilde{z}_x^2(k)}{S_{xx}(k)}, \quad \tilde{v}_2(k) = \frac{\tilde{z}_y^2(k)}{S_{yy}(k)} \quad (9)$$

where $\tilde{z}_x(k)$ and $\tilde{z}_y(k)$ are the components of the innovation vector $\tilde{z}(k) = z(k) - H\hat{X}(k/k)$, and $S_{xx}(k)$ and $S_{yy}(k)$ are the diagonal elements of the covariance matrix

$$S(k) = HP(k/k-1)H^T + R,$$

$$\tilde{v}_3(k) = \alpha_{LT}(k) - \alpha_{LT}(k-1) \quad (10)$$

where $\alpha_{LT}(k)$ and $\alpha_{LT}(k-1)$ are the heading estimates computed by the method of least triangles from using three past data points,

$$\tilde{v}_4(k) = H_{rel}(k) = \frac{\alpha_{LT}(k)}{\theta(k)} \quad (11)$$

where $\theta(k) = \tan^{-1}(y/x)$ is the bearing angle,

$$\begin{aligned} \tilde{v}_5(k) &= \Delta x(k) = x(k) - x(k-1) \\ \tilde{v}_6(k) &= \Delta y(k) = y(k) - y(k-1) \end{aligned} \quad (12)$$

and

$$\tilde{v}_7(k) = \frac{1}{\sigma_{fd}^2} [f_d(k) - f_d(k-1)], \quad (13)$$

where $f_d(k) - f_d(k-1)$ denotes the Doppler shift and σ_{fd}^2 is its variance. For a discussion of the significance of using these features in the neural network training for tracking applications, one may refer to [1,2,5].

3 OPTIMIZATION OF ARCHITECTURE AND TRAINING

3.1 Neural Network Training By Simplex Algorithm

Perhaps the most significant characteristic that enables a neural network to serve as a useful computational device is its learning capability. Implementation of an appropriately tailored learning algorithm, i.e. a rule for adjustment of the network parameters (specifically the interconnection weights) can endow the network the ability to develop the needed structure to result in a corresponding desired computation. The knowledge acquired by the network during this learning is stored in the set of weights.

A number of alternate procedures exist for training a neural network with the available data and different training algorithms usually yield different sets of interconnection weights. While the error backpropagation approach is perhaps the most popular approach for training multilayer neural networks, it has a few shortcomings as well. The backpropagation approach, being a gradient-based search algorithm, is sensitive to the initial starting point (i.e. preliminary selection of weights to start the algorithm). Also because of the gradient-based search property, it is normally trapped by the first optimal point reached and has the tendency to converge to a local minimum. This is generally undesirable since it implies that the knowledge acquired by the network is not optimal. To counter this problem, modified backpropagation algorithms have been developed which include a momentum term that can kick the parameters out of sub-optimal solutions. However, with these algorithms one has to fiddle around with the momentum term and hope that, with the selected starting point, a globally optimal solution can be reached. In general, there is no guarantee of achieving a global optimum.

In our quest to improve the efficiency of neural network learning, which we believe is critical in equipping the network with the knowledge required for reliably recognizing complex target maneuvers, we have implemented an algorithm that employs concepts from simplex optimization and is conducted by splitting the 3-layer neural network into two portions - a linear portion and a nonlinear portion. The connections between the input layer and the hidden layer form the nonlinear portion, while the connections between the hidden layer and the output layer constitute the linear portion. The simplex optimization method is used to find the optimal weights in the nonlinear portion, while a linear least squares minimization is used to determine the optimal weights in

the linear portion of the network. For implementation in the present context, the algorithm can be designed with two distinct stopping criteria. The search for the weights in a specified network structure can be terminated either when the size of the simplex is smaller than a prespecified threshold or the number of iterations performed exceeds a preset threshold.

Implementing the simplex algorithm described above with multiple restart operation (i.e. reinitializing the simplex and executing the algorithm on the new simplex points) has global search property and hence prevents the training procedure to be trapped by local minima of the error function. Furthermore, multiple restarts of the simplex search each time a convergence to a small cluster is attained, guarantees that the procedure will find a globally optimal solution with probability approaching 1.0.

3.2 Network Growing for Optimal Size

The accuracy with which a neural network models a certain process characterized by an input-output mapping or recognizes a set of input patterns depends on a number of factors, the principal one being the size of the hidden layer (or layers in a network which is configured with more than three layers). Only general guidelines however exist for arriving at the optimal architecture to be used in a given application. The more complex the input-output mapping to be approximated, the larger is the set of hidden nodes required, which determines the network size. Employing a larger sized network than necessary has its own drawbacks in that while such a network can learn the input-output mapping presented in the training data, it will attempt to memorize the training patterns used and has poor generalization abilities.

In arriving at a network architecture of optimal size for a given application, two approaches are generally possible. One is to start with a larger number of hidden nodes than necessary and later prune the network by removing redundant nodes. The other is to start with a small sized network initially (with the least number of hidden nodes, for instance) and to progressively grow until a desired degree of accuracy in modeling is achieved. Both of these approaches have been used by various groups of researchers in tailoring an optimal sized network for the specific application at hand.

In our present application in training the network to recognize target maneuvers, we have chosen to use the latter approach for a number of reasons, the principal ones being the following. First of all, the task of training here is a significant one due to the number of feature vectors that are used for obtaining an adequate representation of complex maneuvers. Consequently, the former approach of starting with a network size larger than required can result in unnecessary increased training complexity, with increased learning times and cost particularly at the initial stages. Secondly, and more importantly, a systematic network growing approach can be built into the overall

training algorithm with a convergence condition (stopping rule) being declared when the optimal values of the weights in the correct sized network are obtained.

Such a network growing approach can be integrated with the simplex algorithm described in the last section. For implementation in estimation of maneuver parameters, one starts with the simplest network architecture with one node in the hidden layer, while the input layer comprises of a number of nodes equal to the number of features used for training and the output layer comprises of a number of nodes equal to the number of maneuver parameters to be estimated (which are in turn input to the Kalman filter algorithm). The simplex algorithm is then executed to find the best weights for this structure. Once the weights are found, the mean square error (MSE) associated with this structure is computed and stored together with its weights. The network is then allowed to grow its hidden layer by adding one node and the simplex algorithm is executed once again with the same training data as before. Once the weights for the new structure are found, the MSE for this structure is computed and compared to the stored MSE for the previous structure. If the new MSE is smaller than the previously stored MSE by a preset value, the new structure together with its weights and MSE are stored replacing the previous values. The network is then grown by an additional hidden node and the entire process is repeated. If at any stage of this process, the new MSE is worse than the previously stored MSE or is not better by a preset value, then an optimal structure is claimed to have been found and the training is terminated. A multiple restart of the simplex search can be executed as a part of the overall training process in order to ensure attaining a global minimum of the objective function and hence optimize the training efficiency.

4. TRACKING PERFORMANCE EVALUATION

To perform validation studies that confirm the efficiency of the training scheme used, several tracking experiments were conducted. The following parameter values were employed for the simulations:

- (i) Radar scan period = 5 seconds
- (ii) Standard deviation of range = 100 meters
- (iii) Standard deviation of angle = 0.0003 radian
- (iv) Doppler radar wavelength = 8.57×10^{-3} meter

In order to evaluate any possible degradation in tracking performance due to clutter, simulations of both clutterless and clutter environments were made. The primary difference between the two cases is the use of a standard Kalman filter for the clutterless environment, whereas a Probabilistic Data Association Filter (PDAF) is used to replace this in scenarios that include clutter [4]. For simulations of tracking experiments in clutter environments, the following parameter values were used:

- (i) Spatial clutter density = 0.0009
- (ii) Validation gate size = 16 (Chi-square distribution)
- (iii) Probability of detection = 0.9

(iv) Probability of target inside gate = 0.9997

It may be noted that the choice of the validation gate corresponds to a rather heavy clutter environment. Due to page limitation, in the following we shall only briefly present results from two experiments.

Experiment 1:

The primary goal in this experiment was to evaluate the tracking performance under a combination of maneuvers executed by the target in quick succession. With the target initially located at $(1.5 \times 10^3 \text{ m}, 1.5 \times 10^3 \text{ m})$ and moving at a constant velocity of 250m/sec while maintaining 45° heading, a combination of two maneuvers were executed. The first maneuver occurs at the 6th scan (i.e. $t=30$ sec) and consists of a 5m/sec^2 acceleration which lasts for 1 scan period. The second maneuver consisting of a 10m/sec^2 acceleration occurs at the 9th scan (i.e. $t=45$ sec) and lasts for 2 scan periods. After the second maneuver, the target resumes the constant velocity flight path. To further provide a challenging environment, clutter with spatial density 0.0009 was added at the onset of the first maneuver and was maintained until the end of the experiment. The plots depicting the resulting performance are given in Fig. 3a-c. While Fig. 3a shows a remarkable correspondence between the true track and the estimated track, it can be seen from Fig. 3c that the maneuvers executed by the target are detected and estimated accurately within one sampling period. The position error plot in Fig. 3b indicates that the corrected position estimates are very close to the true target position, while significant errors can result due to the maneuvers when left uncorrected (see the "uncorrected error" entries in Fig. 3b). It is rather well known that when a combination of different maneuvers as in the present experiment takes place, classical tracking methods (such as those based on an Input Estimation Approach [4]) generally fail to deliver acceptable performance due to the fact that compensation for the first maneuver may not be completed before the second maneuver gets initiated. The reason for this shortcoming is that these methods require one to go back a certain number of sampling periods (depending on the window length selected for the computation of the so called "propagation matrix" [4]) in order to compute the correction amounts to be applied.

Experiment 2:

In this experiment, the tracking performance of the neural network scheme was tested when the target executed a sharp 360° turn maneuver. A comparison of the performance was also conducted with that expected from the Interacting Multiple Model (IMM) algorithm [4] which has recently emerged as the most powerful tracking algorithm for tracking target motions that involve turns. For training the neural network, a set of training vectors comprising of the seven input features and the two outputs

$(\hat{\delta}_x \text{ and } \hat{\delta}_y)$ were generated by having the target execute turn maneuvers at various turn rates ranging from 0deg/sec to 12deg/sec at increments of 1 deg/sec, and at three different velocities 200m/sec, 400m/sec and 600m/sec. The test scenario consists of a target initially detected at $(5.0 \times 10^3 \text{ m}, 2.0 \times 10^3 \text{ m})$ with an initial velocity of 500m/sec traveling at an angle of zero degree with respect to the horizon until the 8th scan at which a coordinated turn of 12deg/sec was made for the next 6 scans. At the end of this turn period (6 scans = 30 seconds), the target will have completed a 360° turn, after which it resumes its straight path motion. The target is maintained under observation for a total of 26 scans. The tracking performance delivered by the neural network scheme is shown in Fig. 4a which demonstrates that the tracking scheme keeps up with the target quite well. For a closer evaluation, the behavior of the tracking error in the x-position coordinate as the number of scans increases is shown in Fig. 4b. For comparison purposes, the tracking error expected from a three model IMM algorithm is also plotted in Fig. 4b, which indicates a superior tracking performance by the present scheme. A particular note should be made of the target velocity of 300m/sec during the maneuver, which is different from the velocities included in the neural network training. The level of performance delivered by the neural network attests to the interpolation ability of the network.

Acknowledgement: This research was supported by a grant from the office of Naval research (ONR N00014-95-1-1224).

REFERENCES

1. M.K. Sundareshan and F. Amoozegar, "Neural network fusion capabilities for efficient implementation of tracking algorithms", *Optical Engineering*, Vol. 36, pp. 692-707, March 1997.
2. F. Amoozegar and M.K. Sundareshan, "Constant false alarm rate target detection in clutter: A neural processing algorithm", *Proc. SPIE Int. Symposium on Optical Engineering and Photonics in Aerospace Sensing (Applications of Artificial Neural Networks)*, Orlando, FL, April 1994.
3. D.D. Sworder, "Image enhanced tracking", *IEEE Trans. Aerospace and Electronic Systems*, Vol. AES-25, pp. 701-710, 1989.
4. Y. Bar-Shalom and X.R. Li, *Estimation and Tracking*, Artech House: 1993.
5. M.K. Sundareshan and Y.C. Wong, "Neural network schemes for data fusion and tracking of maneuvering targets", *Performance Report 3 (ONR # N00014-95-1-1224)*, University of Arizona, Engineering Experiment Station, Tucson, March 1997.
6. A.P. Andrews, M.S. Greewal, *Kalman Filtering Theory and Practice*, Prentice Hall Information and System Science Series, 1993.

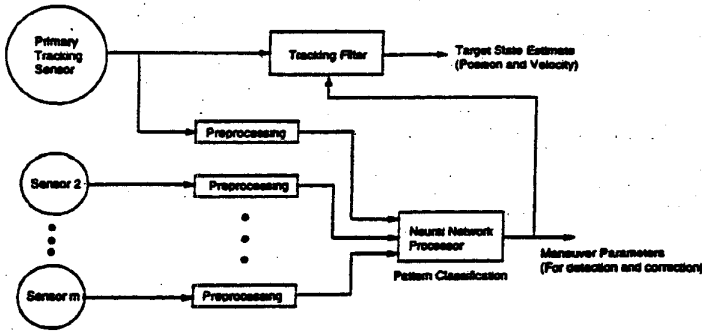


Fig. 1. Data fusion architecture for neural network-based tracking

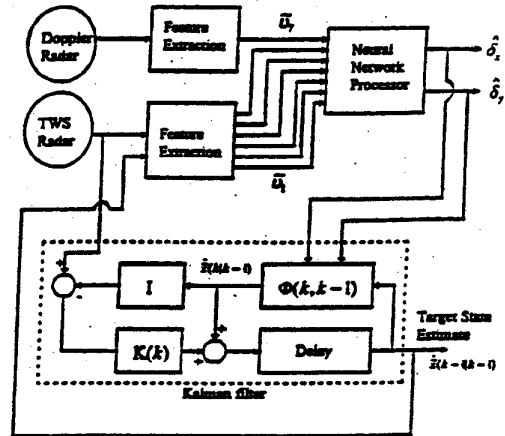


Fig. 2. Tracking architecture depicting integration of neural network and Kalman filter

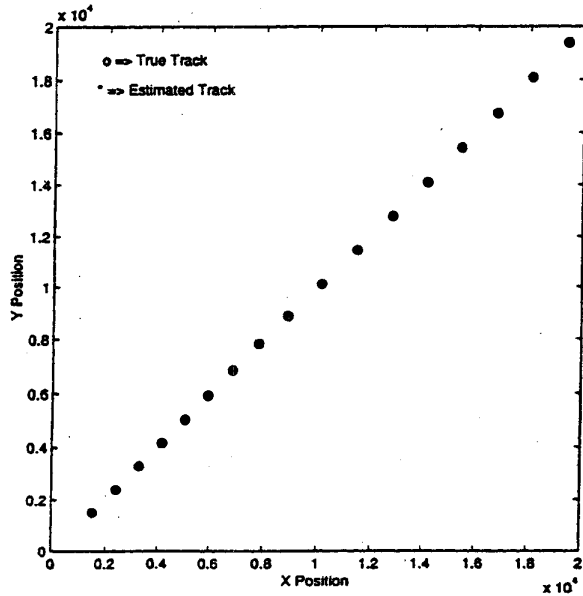


Fig. 3a. Tracking performance in Experiment 1

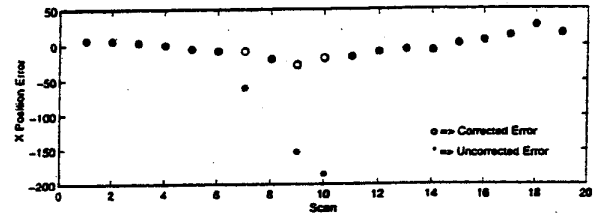


Fig. 3b. Position estimation error plot

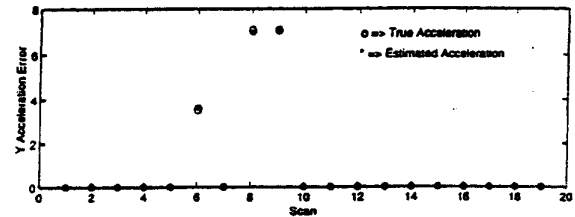


Fig. 3c. Estimation of target accelerations

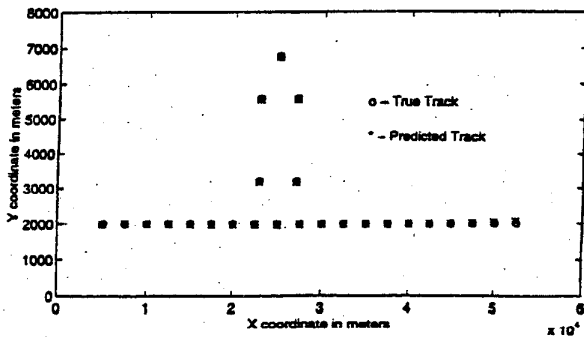


Fig. 4a. Tracking performance in Experiment 2

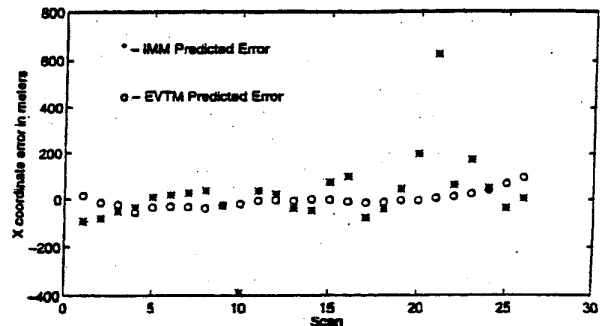


Fig. 4b. Tracking error comparison with IMM algorithm